

Frontend presentation

How GuardMe Dashboard works

GuardMe Dashboard is an Angular SPA that talks to the NestJS gateway on port 3000. It is the operator console for the personal secure web gateway: login, live monitoring, traffic/security history, firewall rules, encrypted vault, and analytics.

Stack: Angular components, NgRx (state + side effects), Angular Material, HttpClient, Socket.IO client. No JWT in localStorage — the session lives in an HttpOnly cookie sent automatically on API and WebSocket calls.

Browser loads SPA



appInit → GET /auth/me (restore session from cookie)



authGuard / guestGuard (wait until auth initialized)



Authenticated shell → lazy-loaded feature routes



Parallel paths:

- REST → traffic history, security history, rules CRUD, vault, analytics
- WebSocket (Socket.IO /events) → live traffic logs, security events, system health



NgRx store (realtime slices) → Dashboard live feed + stat cards

Mode	Where used	Source
Live buffer	Dashboard overview	WebSocket → NgRx (last 100 traffic + security events)
Paginated history	Traffic & Security pages	REST GET /siem/traffic-logs and GET /siem/security-events

Security review:

- **HttpOnly session cookie** — no tokens in localStorage; credentialsInterceptor sets withCredentials: true only for API URLs.
- **Open-redirect protection** — sanitizeReturnUrl() only allows same-app relative paths (/dashboard, not //evil.com).
- 401 handling — authErrorInterceptor distinguishes re-auth required vs session expired vs invalid password; login/register/me/verify-password 401s do not trigger logout.
- **Re-authentication** — 15-minute window from lastAuthAt; proactive dialog + toolbar “Re-auth needed” chip + reactive 401 from sensitive APIs.
- **Vault passwords never in NgRx** — reveal action passes password straight to a dialog; reducer does not persist plaintext.
- **Generic login errors** — no “user not found” vs “wrong password” distinction in the UI (matches backend timing-safe auth).

Known tradeoffs / future work:

- Live vs history split — Dashboard shows the last 100 realtime events; full history requires the Traffic/Security pages.
- History refetch — Traffic/Security refetch on every filter/page change; no NgRx cache
- Replace window.confirm on rule/vault delete with Material confirm dialog.

///// FILES EXPLANATION /////

➤ [src/index.html](#)

The application's entry HTML page. Only HTML page that is actually served by the web server. It contains material fonts/icons and the root Angular component (*app-root*), after which Angular dynamically renders the entire dashboard without reloading the page.

Angular is a Single Page Application (SPA), so after the page loads, Angular completely controls everything inside.

➤ **src/main.ts**

Single entry point for the Angular SPA. Angular boots directly from the root App component and use the configuration defined in *app.config.ts*. Instead of *AppModule* (older Angular versions).

➤ **src/environments**

➤ **environment.model.ts**

Defines production, *apiBaseUrl*, *wsUrl*, *reAuthTimeoutMinutes* (minutes after login/verify-password before sensitive actions require re-auth).

➤ **environment.ts (dev)**

http[:]//localhost:3000 API, *http[:]//localhost:3000/events WebSocket*, 15-minute re-auth window

➤ **environment.production.ts**

➤ **src/app**

➤ **app.ts:**

Root Angular component serves as the application's container — only *<router-outlet>*, meaning display whichever page the router chooses. No layout logic here, shell is a routed child.

The router outlet is a placeholder where Angular dynamically loads the active page component based on the current route.

➤ **app.routes.ts:**

Top-level routes: *'/login'* and *'/register'* (guest-only: protected with *guestGuard* meaning only unauthenticated users can access it), authenticated shell at *'/'* , protected with *authGuard*, meaning everything inside requires authentication.

Every page of the app appears inside shell component which contains sidebar, header and content. Content is changing based on child routes (dashboard, traffic, security, rules, vault, analytics).

Everything is lazy loaded which means Angular loads each feature only when needed.

Redirection wildcard → dashboard catches unknown URLs. Even if the user is not authenticated *authGuard* will trigger redirect to login page.

➤ **app.config.ts:**

It acts as the central configuration point during application startup. It registers application providers:

- routing - loads routing configuration

- HTTP communication (client + interceptors) - every HTTP request passes through custom interceptor (attach JWT token, error handler, refresh token?)

- animations

- state management (NgRx store/effects)

- localization (en-GB locale) - registers British English locale (for dates dd/mm/yyyy)

- API bindings - dependency injection for backend API services

- authentication initialization - *aplnit* dispatcher on startup, before user interacts with the app, initialize authentication (session restore, check session?)

➤ **core** - Cross-cutting infrastructure shared by all features.

➤ **api**

Component → Feature Store / Service → Abstract API → HTTP API → HttpClient → Backend

Abstract API classes define the application's data access contracts. Components depend only on these

contracts rather than specific HTTP implementations, improving modularity.

- api.providers.ts:
 - Registers concrete HTTP implementations for the abstract API contracts using Angular dependency injection.
 - binds AuthApi → HttpAuthApi, SiemApi → HttpSiemApi, RulesApi → HttpRulesApi, VaultApi → HttpVaultApi, AnalyticsApi → HttpAnalyticsApi, RealtimeApi → SocketRealtimeApi.
- analytics.api.ts / http/http-analytics.api.ts:
 - /siem/analytics/summary
 - Sends analytics requests to the backend and converts filter values into HTTP query parameters.
- auth.api.ts / http/http-auth.api.ts:
 - register, login, logout, profile, verify-password against /auth/*.
 - Handles user authentication, profile retrieval, logout, and password verification while mapping backend DTOs into frontend models.
- realtime.api.ts / http/socket-realtime.api.ts:
 - Maintains the WebSocket connection, listens for live security events, traffic logs, and system status updates, then exposes them as RxJS observables.
 - Socket.IO connect/disconnect and observables for traffic, security events, system status.
 - Reconnect with exponential backoff (1s–15s).
- rules.api.ts / http/http-rules.api.ts:
 - Provides CRUD operations for user firewall rules (list, create, update, delete rules).
- siem.api.ts / http/http-siem.api.ts:
 - /siem/traffic-logs; /siem/security-events
 - Retrieves paginated traffic logs and security events with optional filtering (converting filter values into query parameters).
- vault.api.ts / http/http-vault.api.ts:
 - vault status, unlock/lock, credential CRUD, reveal password
 - Communicates with the secure password vault for locking, unlocking, and managing encrypted credentials.
- http/auth-api.mapper.ts — map backend DTOs to frontend domain models.
- http/siem-api.mapper.ts — map backend DTOs to frontend domain models.
- http/http-query.util.ts — builds query params, omitting empty filter values.
- http/websocket-events.ts — event name constants mirroring backend (TRAFFIC_LOG, SECURITY_EVENT, SYSTEM_STATUS).
- guards
 - auth.guard.ts:
 - This guard protects private routes. It waits until authentication is initialized and only allows access to authenticated users, otherwise redirecting them to the login page.
 - guest.guard.ts:
 - This guard prevents authenticated users from opening pages like login or registration by redirecting them back into the application sanitized returnUrl (default /dashboard).
- interceptors
 - credentials.interceptor.ts:
 - Sets withCredentials: true for requests to environment.apiUrl so HttpOnly cookies are sent.
 - Automatically attaches HttpOnly session cookies to every API request while avoiding sending credentials to third-party URLs. This interceptor automatically includes secure session cookies on API requests.
 - auth-error.interceptor.ts:
 - On 401 from non-bootstrap API calls: dispatches reauthRequired or sessionExpired based on error body; rethrows error for feature-level handling.
 - Intercepts API authentication errors and dispatches NgRx authentication actions when the user's session expires or reauthentication is required. This interceptor provides centralized authentication error handling. Instead of every component checking for session expiration, it dispatches authentication actions whenever the backend returns a 401 response.

- index.ts:
 - Registers interceptors in order: credentials, then auth-error.
- **models**
 - Domain types: UserProfile, TrafficLog, SecurityEvent, FirewallRule, VaultCredential, analytics summaries, SystemStatus, pagination, activity feed items.
- **utils**
 - **analytics helpers**
 - analytics-chart.util.ts:
 - Builds chart structures for analytics page (CSS bar charts and dimensions). It also calculates axis scales and percentages so charts remain proportional to amount of data.
 - analytics-query.util.ts:
 - Converts filter values from the UI into clean API query parameters, removing empty dates before sending requests.
 - **authentication helpers**
 - auth-error.util.ts:
 - Hides HTTP backend error details and converts them into user-friendly authentication messages; detects re-auth requirements / session expiration and invalid-password 401s.
 - re-auth.util.ts:
 - Compares lastAuthAt to reAuthTimeoutMinutes.
 - **date/time manipulation**
 - datetime-filter.util.ts:
 - Standardizes date and time handling by converting between user input and ISO timestamps required by the backend.
 - **filter/query builders** - Sanitizes filter input values → API query objects for traffic logs and security events.
 - history-query.util.ts
 - analytics-query.util.ts
 - **HTTP error parsing**
 - http-error.util.ts:
 - Provides a single way to read backend error messages regardless of how the server formats the response.
 - **formatting**
 - ip-display.util.ts
 - Normalizes IP addresses for display in plain IPv4 form.
 - **security helpers**
 - return-url.util.ts:
 - Open-redirect protection for post-login navigation, allowing only internal application routes as return URLs.
 - **traffic statistics**
 - traffic-verdict.util.ts:
 - Standardizes verdict and policy decision labels mapping them to CSS classes for consistency throughout the application.
 - traffic-stats.util.ts:
 - Verdict total and allowed/blocked counts calculated of traffic logs.
 - **URL formatting**
 - url-display.util.ts
 - Shortens long URLs so they fit cleanly inside tables and dashboard layouts
 - **Error helpers** - feature-specific error mapping with re-auth awareness
 - vault-error.util.ts
 - rules-error.util.ts
 - **validators/auth.validators.ts** - username pattern/length, password length, confirm-password match, normalizeUsername()
- **features** - The application is organized into independent lazy-loaded feature modules, where each feature encapsulates its own user interface, routing, and interactions with the NgRx store. Each feature has: *.component.html, *.component.scss, *.component.ts, *.routes.ts.

- **auth**
 - login/login.component.ts
The login component uses reactive forms with centralized validation and dispatches authentication actions (*AuthActions.login* with optional returnUrl) through NgRx while avoiding direct communication with backend services.
Instead of inline validators, everything comes from *auth.validators.ts*.
Clears password field after submit.
 - register/register.component.ts
Register form with confirm password; effect auto login on success.
 - auth-form.scss — shared auth page styling.
- **dashboard**
The dashboard provides a real-time overview of network traffic and security activity, displaying system health (DB, VirusTotal), traffic statistics (allowed/blocked/traffic/security stat cards), verdict distribution, and a live merged activity feed via LogListComponent. It uses NgRx selectors to automatically update the UI whenever new data is received. Includes proxy setup instructions for generating test traffic.
- **traffic**
The traffic history page shows all monitored network requests with filtering, pagination, and page-level traffic statistics. Each log includes request details such as source and destination IPs, URL, policy decision, threat verdict, and risk score. So fields in filter bar are: decision, verdict, URL, destination IP, date range.
Direct API loading. Server-paginated history table (15 per page) via SiemApi - not from NgRx realtime store.
- **security**
The security history page displays security events in a paginated, filterable list, allowing users to search by event type, severity, and time range. Each event can be expanded to view additional metadata for easier investigation (hides userId / ruleId).
Direct API loading. Server-paginated history table (15 per page).
- **analytics**
The analytics page visualizes aggregated traffic and security statistics using reactive NgRx selectors (date-range filters → NgRx loadSummary), while all chart preparation and business calculations are performed outside the component (CSS-based charts for policy decisions, verdicts, severities, event types, time buckets, top hosts).
Component doesn't calculate analytics, it goes from store -> selectors -> component -> template
OnPush use: changeDetection: ChangeDetectionStrategy.OnPush
Shows system status snapshot from analytics load.
- **rules**
The rules page displays system read-only rules and allows users to manage custom allow/block rules. Users can create, edit, enable or disable, and delete rules through a dialog with built-in validation via RuleFormDialogComponent.
//suggestions: on delete: Material confirm dialog instead of window.confirm.
 - rule-form-dialog — create/edit firewall rule modal.
The Rule Form Dialog provides a simple interface for creating and editing firewall rules. It validates user input and supports domain and IP-based rules with configurable actions (allow/block).
CRUD operations; NgRx state.
- **vault**
The password vault securely stores encrypted credentials that can only be accessed after unlocking the vault with the user's account password. Users can add, edit, delete, and reveal saved credentials. Lock button clears server-side vault key.
Credential CRUD; NgRx state; viewPassword opens ViewPasswordDialogComponent (password only in dialog memory).
 - credential-form-dialog — add/edit credential (service name, username, password).
 - view-password-dialog — securely view a decrypted password after unlocking the vault (one-time password display after reveal API call)
- **layout/shell**

Authenticated app layout: The Shell component provides the common layout used after login, including the navigation Material sidenav, toolbar, user information, and router outlet for feature pages.

➤ shell.component.ts

Nav items (Dashboard, Traffic, Rules, Security, Analytics, Vault); toolbar shows username, “Re-auth needed” chip when stale, Sign out; hosts child `<router-outlet>`.

The Shell only handles: navigation, logout, opening reauthentication and displaying username.

It doesn't contain business logic. Functions doesn't call API but dispatches NgRx actions.

Modern Angular: `readonly username = toSignal(...)` instead of `username$ = ...`

➤ shell.component.html / shell.component.scss — layout structure and styling.

➤ shared

➤ components

➤ **filter-bar**

Config-driven filters (select, text, datetime); grid/split/compact layouts; Apply/Clear emits FilterValues. Used on Traffic, Security, Analytics.

Generates a dynamic filtering form (filtering controls) from configuration objects, supporting text fields, select lists, and date-time inputs while emitting normalized filter values, allowing multiple pages to reuse the same filtering component with different fields.

The application uses reusable components such as the Filter Bar, Empty State, Status Badge, Stat Card, and Log List to maintain a consistent interface across different pages. These components reduce code duplication and improve maintainability.

Instead of TrafficFilterComponent, SecurityFilterComponent, AnalyticsFilterComponent there is FilterBar + configuration.

Modern angular use: `input()`, `output()`, `computed()`, `OnPush`

➤ **reauth-dialog**

Modal password confirmation; dispatches `verifyPassword`; Cancel closes without verifying.

Prompts the user to re-enter their password again before performing sensitive operations such as accessing the password vault; and dispatches password verification requests.

➤ **log-list**

Renders merged activity feed items with icons and verdict styling (Dashboard).

Displays a live activity feed by displaying traffic logs and security events in a unified chronological list, using different layouts depending on the event type.

➤ **stat-card**

Metric card with icon and accent color.

Provides a consistent visual representation of dashboard statistics across the application using configurable labels, values, icons, and colors.

➤ **status-badge**

Health chip (ok/degraded/unknown) for DB and VirusTotal.

Displays the health state of backend services using color-coded status chips.

➤ **empty-state**

Placeholder when lists are empty.

This reusable component provides a consistent empty-state message and icon whenever a page has no content to display. Displays a reusable placeholder when no data is available.

➤ models

➤ filter-bar.model.ts — FilterFieldConfig, FilterValues types.

➤ styles — Shared styles provide reusable visual patterns such as verdict colors and history page layouts, ensuring a consistent appearance across multiple screens.

➤ verdict.scss

➤ history-page.scss

➤ store - NgRx state management. The application follows a feature-based NgRx architecture where each business module manages its own state, actions, reducers, effects and selectors independently. No separate dashboard or realtime reducer (dashboard uses composed selectors; realtime is effects-only).

➤ analytics

The analytics module retrieves aggregated traffic and security statistics through NgRx Effects and exposes data using selectors. This keeps business logic outside the UI and minimizes redundant API

- requests.
- analytics.actions.ts:
 - loadSummary, loadSummarySuccess, loadSummaryFailure, clearError
- analytics.effects.ts:
 - Fetches summary; snapshots current systemStatus from store at load time. Analytics data is loaded asynchronously through NgRx Effects, combining API responses with the current system status stored in the application state to minimize unnecessary network requests.
- analytics.reducer.ts:
 - The reducer stores only essential analytics information, including the latest query, loading state and cached results, keeping the application state minimal.
 - State: summary, systemStatus, lastQuery, loading, error.
- analytics.selectors.ts:
 - Selectors transform raw analytics data into presentation-ready chart datasets, time buckets, top hosts, ensuring components remain simple and free of data transformation logic.
- auth
 - Authentication is implemented using an event-driven NgRx architecture with login, logout, session expiration, and password re-verification handled consistently through NgRx actions. Sensitive operations require recent authentication/password re-verification, providing an additional security layer before allowing access to protected resources.
 - auth.actions.ts:
 - login, register, logout, loadProfile, sessionExpired, reauthRequired, verifyPassword
 - auth.effects.ts:
 - API calls.
 - Login: *exhaustMap()* - prevents double-click login.
 - Register: register -> login automatically.
 - Logout: *logoutSuccess* happens even if API fails, logout -> /login.
 - The application continuously monitors authentication freshness and automatically prompts user for password verification before sensitive actions.
 - Re-auth timer: interval(30000) -> check lastAuthAt -> if stale -> open dialog. Periodic stale check (30s); opens *ReauthDialogComponent* on *reauthRequired*; closes dialog on verify success. Dialog handling: to avoid multiple dialogs checks *alreadyOpen* flag.
 - Navigation effects: No navigation inside components but in effects. Post-login navigation with sanitized *returnUrl*.
 - auth.reducer.ts:
 - State: user, status, error, initialized, verifyPasswordLoading, verifyPasswordError, reAuthStaleTick.
 - Authentication state stores only user identity and session metadata, while all authentication logic remains inside NgRx reducers and effects.
 - auth.selectors.ts:
 - selectAuthState, selectAuthUser, selectUsername, selectLastAuthAt, selectReAuthStaleTick, selectIsReAuthStale, selectIsAuthenticated, selectAuthInitialized, selectAuthLoading, selectAuthError, selectVerifyPasswordLoading, selectVerifyPasswordError
- dashboard
 - The dashboard combines traffic logs and security events into a single chronological activity feed, giving administrator a unified overview of live network activity.
 - dashboard.selectors.ts:
 - selectActivityFeed* merges realtime traffic + security events into a sorted *ActivityItem[]* for the dashboard page. No reducer.
- realtime
 - The application establishes a real-time connection after successful authentication to receive live traffic logs, security events, and system status updates, enabling the dashboard to display live security information. The connection is automatically terminated and related data cleared when the user logs out.
 - Lifecycle: Authentication starts -> connect websocket -> listen -> disconnect -> clear store
 - realtime.effects.ts:
 - Connects Socket.IO on login/profile success; disconnects on logout/session expiry.

Merges traffic + security streams into insert/update actions; streams system status; clears all on logout.

➤ **rules**

Firewall rules are managed through NgRx Entity, enabling efficient CRUD operations while keeping system policies separated from user-defined rules.

States: systemRules, userRules, loading, saving, loaded, error.

- rules.actions.ts — load, create, update, delete rules.
- rules.effects.ts — HTTP via RulesApi. One action -> one API call.
- rules.reducer.ts — system rules array + user rules entity adapter.
- rules.selectors.ts — combined system + user rules for the UI.

➤ **security-events**

Only for real-time displaying. Security events are stored as normalized entities in-memory collection.

- security-events.actions.ts — insertEvent, updateEvent, clearEvents (realtime only).
- security-events.reducer.ts — NgRx EntityAdapter, max 100 entities, sorted by timestamp.
- security-events.selectors.ts — all security events, entities, total.

No effects — covered by *RealtimeEffects*.

➤ **system-status**

The system status module maintains the latest health information of gateway availability (database + VirusTotal API) using a state object that is continuously updated through real-time events.

- system-status.reducer.ts — *current: SystemStatus | null*.
Updated by *RealtimeEffects.streamSystemStatus\$*; cleared on logout.

Realtime updates overwrite the current status. No history, no duplicated information.

➤ **traffic**

Only for real-time displaying. Traffic logs are normalized and maintained in-memory collection, allowing efficient insertion, updates and real-time statistical summaries. Selectors compute traffic metrics without placing processing logic inside UI components.

- traffic.actions.ts — insertLog, updateLog, clearLogs (realtime only).
- traffic.reducer.ts — NgRx EntityAdapter, max 100 entities, sorted by timestamp.
- traffic.selectors.ts — all logs, verdict stats, allowed/blocked counts.

No effects — covered by *RealtimeEffects*.

➤ **vault**

The password vault stores only credential metadata in the application state, while secret passwords are retrieved only on demand, when explicitly requested and never saved in the NgRx store.

- vault.actions.ts:
status, unlock/lock, credential CRUD, reveal (password not stored in state).
- vault.effects.ts:
All vault HTTP operations. Dispatches *loadCredentials* after status/unlock success.
//suggestions: for create, update, delete instead of *switchMap()* maybe *exhaustMap()* or *concatMap()*.
- vault.reducer.ts:
Locked flag, credential entities, loading/saving/error states. Reveal success does not persist password. The password never exists inside the application state (API -> Action -> Dialog -> Gone).
- vault.selectors.ts:
Locked state, credential list, loading flags.
Unlock flow: Load Status -> Locked? -> Unlock -> Load Credentials
Lock: Everything disappears immediately - *credentials: credentialsAdapter.getInitialState()*
Reveal password: instead of *store.password* It goes directly *dialog.open(...)*